LAB REPORT

Course Code : ICE 3146 **Course Title :** Microprocessor and Interfacing Laboratory
Lab Task No. : 11
Lab Task Topic: Implementation of Scrolling Text on a Dot Matrix Display.

No.	Evaluation Criteria with Marks	Put Tick (✓) Mark					Marks Obtained	Remarks
		Excellent	Good	Fair	Poor	Fail		
REPORT WRITING								
1	Theory & Objectives (2.5)							
2	Materials & Methods (5)							
3	Data Analysis & Discussion/Diagram (10)							
4	Precautions and Conclusion (2.5)							
5	Time Management (5)							
	TOTAL							
LAB PERFORMANCE								
1	Preparation for Lab (5)							
2	Experiment and Engagement (15)							
3	Communication (5)							
	TOTAL							
25th June, 2026 Date of Experiment		01st August, 2026 Date of Submission						_____ Teacher's Signature

Semester : Summer **Year :** 2026 **Level-Term :** L2-T2 **Section :** A2

Submitted to : **Sameer Khairul**
Lecturer, Dept. of Information and Communiction Engineering
Daffodil International University

Submitted by : **Tanzim Ahamed**
Reg.No. : 242-50-036
Dept. of Information and Communication Engineering
Daffodil International University

Title: Implementation of Scrolling Text on a Dot Matrix Display.

Abstract:

This experiment was performed using the Intel 8086 Microprocessor Trainer Kit to generate an animated character pattern on an 8×8 dot-matrix display. Eight font frames were stored in memory and displayed sequentially through the 8255 Programmable Peripheral Interface. In the first seven frames, the central vertical line of the character was extended gradually. In the eighth frame, the complete capital letter **T** was displayed by adding the full horizontal line at the top. Each frame was refreshed repeatedly before the next frame was selected. The experiment demonstrated frame-based animation, dot-matrix scanning, loop control, memory addressing, delay generation, and peripheral interfacing using 8086 assembly language.

Introduction (Theory):

The Intel 8086 is a 16-bit microprocessor that can communicate with external hardware devices through Input/Output (I/O) ports. Devices such as LEDs, seven-segment displays, LCDs, and dot matrix modules are common output peripherals. Connecting these devices with the microprocessor is called interfacing. In assembly language, the OUT instruction is used to transfer data from a register to an output port and thereby control external hardware.

An 8×8 dot-matrix display consists of 64 LEDs arranged in eight rows and eight columns. Characters and graphical patterns are displayed by sending binary pattern data while selecting the display lines sequentially.

In this experiment, eight separate font frames were stored in memory. Each frame contained eight bytes. A binary 0 represented an illuminated LED, while a binary 1 represented an OFF LED. The first frame illuminated only two pixels at the top center of the display. In each successive frame, the central vertical line was extended downward. The final frame displayed a complete capital letter **T** by illuminating the full top row and the two center columns.

Port B was used to send the font bytes, while Port C was used for display-line selection. Rapid scanning produced a stable pattern due to persistence of vision.

Material:

Hardware	Software
1. Intel 8086 Microprocessor Trainer Kit 2. 8×8 Dot Matrix Display Module 3. Interface Connections / Trainer Board 4. Power Supply	1. Assembly Program 2. On-board Monitor / Kit Commands 3. MASM and MDA-8086

]

Method:

Code:

```
CODE SEGMENT
ASSUME CS:CODE, DS:CODE, ES:CODE,
SS:CODE

PPIC_C  EQU 1EH
PPIC    EQU 1CH
PPIB    EQU 1AH
PPIA    EQU 18H

ORG 1000H

MOV AL,10000000B
OUT PPIC_C,AL

MOV AL,11111111B
OUT PPIA,AL

L1:
   MOV SI,OFFSET FONT1
   MOV BL,8

L3:
   MOV BH,30

L2:
   PUSH SI
   CALL SCAN
   POP SI

   DEC BH
   JNZ L2

   ADD SI,8
   DEC BL
   JNZ L3

   JMP L1

SCAN PROC NEAR
   MOV AH,00000001B

SCAN1:
   MOV AL,BYTE PTR CS:[SI]
   OUT PPIB,AL

   MOV AL,AH
   OUT PPIC,AL

   CALL TIMER

   INC SI
   CLC
   ROL AH,1
   JNC SCAN1

   RET
SCAN ENDP

TIMER:
   MOV CX,300

TIMER1:
   NOP
   NOP
   NOP
   NOP
   LOOP TIMER1
   RET

FONT1:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 01111111B
   DB 01111111B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT2:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 00111111B
   DB 00111111B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT3:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 00011111B
   DB 00011111B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT4:
   DB 11111111B
   DB 11111111B
```

```
   DB 11111111B
   DB 00001111B
   DB 00001111B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT5:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 00000111B
   DB 00000111B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT6:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 00000011B
   DB 00000011B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT7:
   DB 11111111B
   DB 11111111B
   DB 11111111B
   DB 00000001B
   DB 00000001B
   DB 11111111B
   DB 11111111B
   DB 11111111B

FONT8:
   DB 01111111B
   DB 01111111B
   DB 01111111B
   DB 00000000B
   DB 00000000B
   DB 01111111B
   DB 01111111B
   DB 01111111B

CODE ENDS
END
```

Execution (PC Mode):

1. First, the assembly language program was written in Notepad and saved as .asm file.
2. Machine code execution using CMD-

   C:\MDA\8086\Asm8086>MASM.exe SUM26A19.asm
   Object filename [SUM26_1.OBJ]: SUM26A19
   Source listing [NUL.LST]: SUM26A19
   Cross-reference [NUL.CRF]:
   0 Warning Errors
   0 Severe Errors

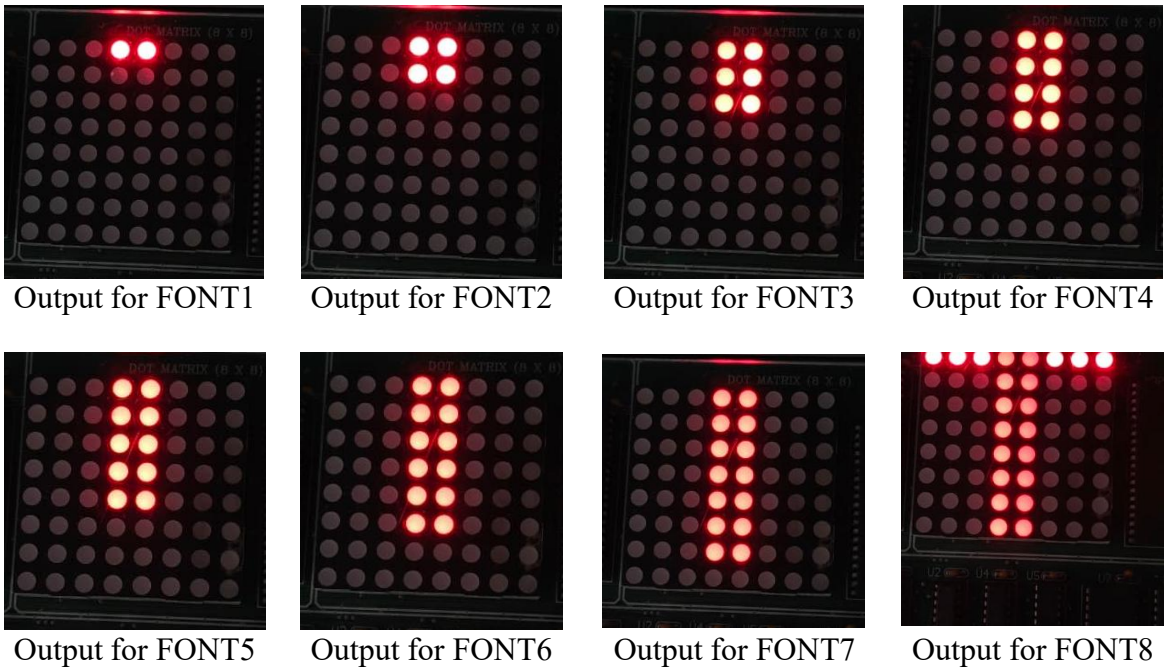
1. Then again the .ABS file is created using the CMD-

   C:\MDA\8086\Asm8086>LOD186 SUM26A19
   Output Object File [C:SUM26A19.ABS]:SUM26A19
   Map Filename [C:NUL.MAP]:SUM26A19

   ***LOAD COMPLETED

2. Then in MDA 8086 from FILE DOWNLOAD the SUM26A19.ABS file opened and Run executed.
3. The output and register values were observed through the simulator window and the DOT Matrix Display shows the output according to the code.

Result:



Output for FONT1 Output for FONT2 Output for FONT3 Output for FONT4

Output for FONT5 Output for FONT6 Output for FONT7 Output for FONT8

Fig 1: DOT Matrix Display outputs for different Sequence of Code.

Observed ROW/Pattern Data**:**

Step	Codes								Font
1	11111111B	11111111B	11111111B	01111111B	01111111B	11111111B	11111111B	11111111B	**T**
2	11111111B	11111111B	11111111B	00111111B	00111111B	11111111B	11111111B	11111111B	
3	11111111B	11111111B	11111111B	00011111B	00011111B	11111111B	11111111B	11111111B	
4	11111111B	11111111B	11111111B	00001111B	00001111B	11111111B	11111111B	11111111B	
5	11111111B	11111111B	11111111B	00000111B	00000111B	11111111B	11111111B	11111111B	
6	11111111B	11111111B	11111111B	00000011B	00000011B	11111111B	11111111B	11111111B	
7	11111111B	11111111B	11111111B	00000001B	00000001B	11111111B	11111111B	11111111B	
8	01111111B	01111111B	01111111B	00000000B	00000000B	01111111B	01111111B	01111111B	

Discussion:

In this experiment, a frame-based animation was created on an 8×8 dot-matrix display using the 8086 microprocessor and 8255 PPI. Port B transferred the font data, while the SCAN procedure and timer refreshed the display. Eight font blocks acted as animation frames, each displayed 30 times before the next frame. The first seven frames gradually built the vertical stem, and the final frame completed the top bar and center stem, forming the capital letter **T**. Thus, the program demonstrates progressive character construction instead of conventional scrolling.